

Title, Goal Statement, and Conventions

Web-Based Interactive Model Visualizer

Goal: Create a web-based Model visualizer that allows users to quickly change layers of the model as well as create models based on images uploaded via AI.

Conventions: The basic input to the system is user login credential and a 2D image to convert /previously saved model. The basic output is the interactive 3D model that's rendered in the user's web browser.

Block Diagram

Conventions:

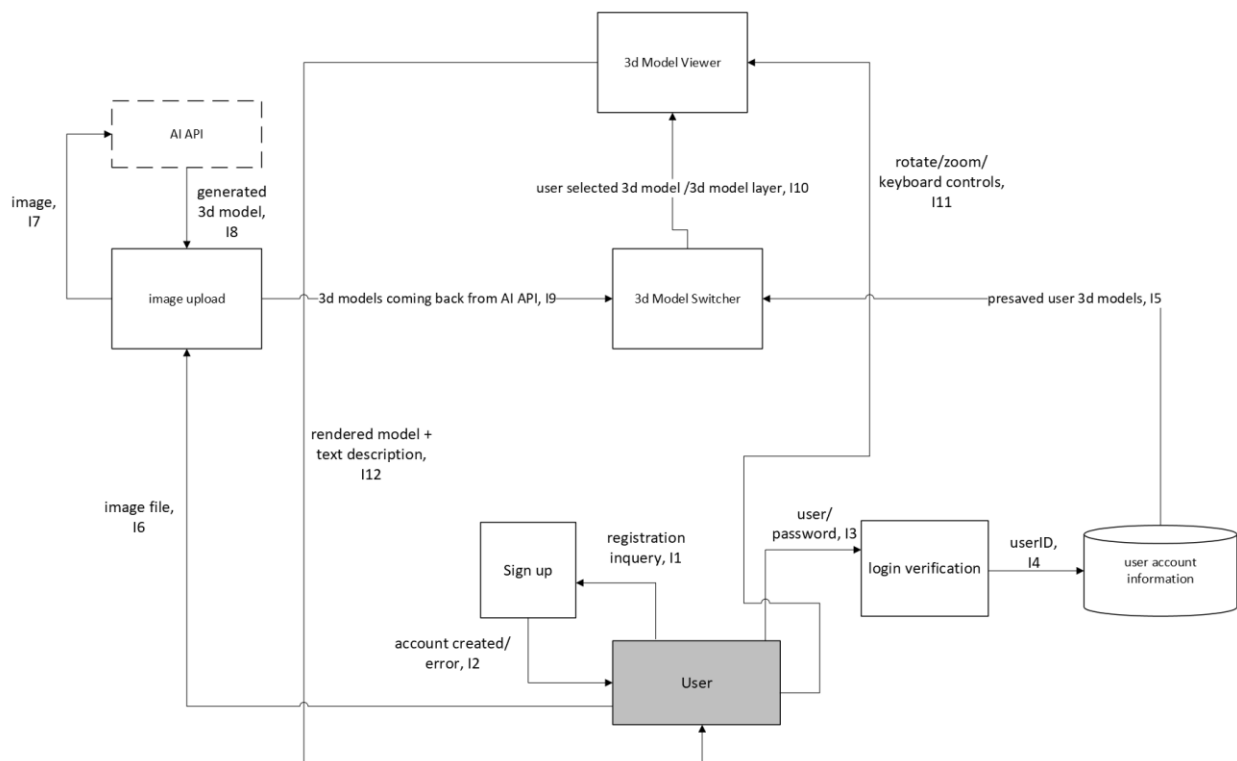
Solid White Box: component being built

Dashed Box: external API (AI image-to-3D)

Gray Box: user, external actor

Cylinder: data stored (user credentials and models)

Arrow: interface



Component Responsibility Table

ID	Component	Responsibility	Interfaces In and Out	Primary Owner
C1	Login/Signup Section	Hidden Stakeholder for Cybersecurity – gives security to the application. Allows saving and user storage to provide individual experiences with their own models	Input: user login information Output: models saved by the user	Veronica Malusky
C2	3d model viewer	Renders the selected 3D model in the browser so the user can inspect it interactively	Inputs: 3D model from the switcher Output: 3D model to the user	Sophia Munoz
C3	3d model layer changer/3d model switcher	Will allow users to change models and their layers quickly and efficiently. Will also work as a connection point between the upload/login components and the 3d model viewer.	Input: 3D Models from the Login and from the image upload. Output: selected model/layer to the 3d model	Jake Martin
C4	Image upload	User will upload the image and in the backend to the AI model, which will output the 3D model.	Input: image upload from user Output: 3d models to the 2d model switcher	Abby Kraft

Interface specification table

Component	Input	Output	Data Format	Protocol	Error	Error Handling
Login -> switcher	User Information obtained by payload	N/A	Payload model of user information	Direct function call (Login button)	Invalid login	Have message pop up display and allow the user to reinput user/password

Upload -> switcher	The upload's output, which is the 3D model	N/A	The 3D model will be in GLB format	Direct function call	Invalid image upload	Popup that displays the image wasn't able to be rendered into a 3D model
Switcher -> viewer	3d models from login and from upload, takes in user input on model to show	Outputs 3d model to the viewer	The 3D model will be in GLB format	Direct function call	Model failed to load	Have 3d model viewer pop up with message saying the model failed to load

Payload example: Payload from Login/Screen

API Request Payload

```
{
  "email": "example@gmail.com",
  "password": "Password1"
}
```

API Response Payload example

```
{
  "FirstName": "FName",
  "LastName": "LName",
  "Models":
    {
      "ID": Primary Key,
      "Title": User Inputted Name,
      "FileInfo": GLB file
    }
}
```

Data-Flow Diagram

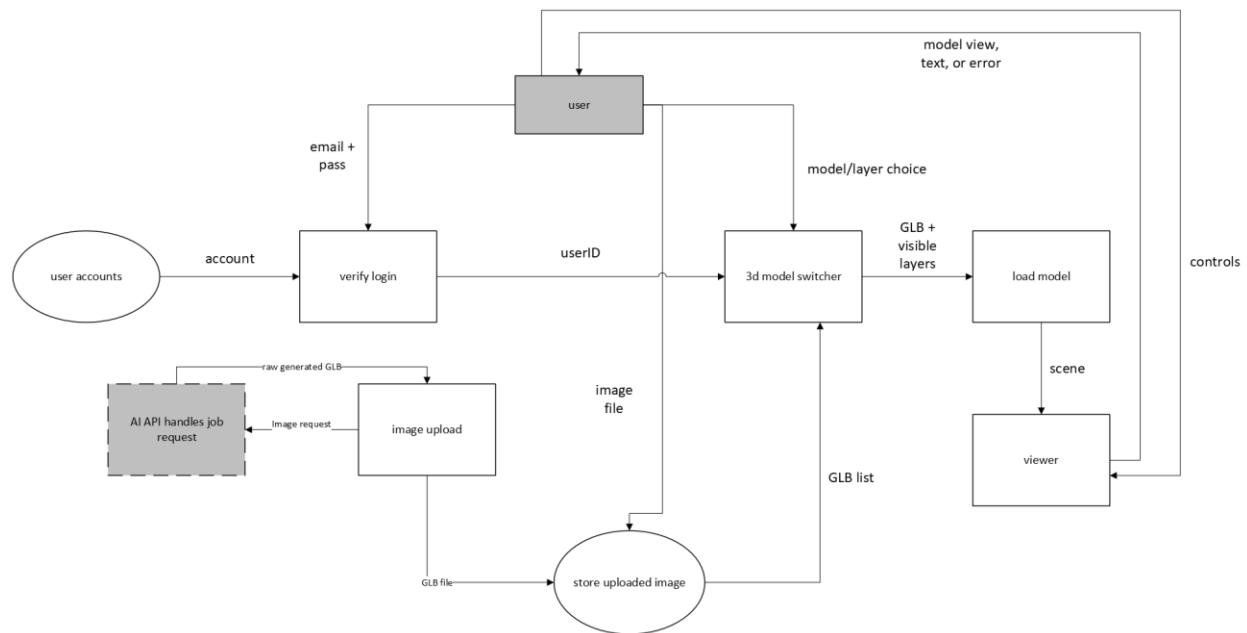
Conventions:

Oval: data stored

Gray box: external entity that sends/receives data

Dashed box: external system

White box: processes



Architecture Pattern and Justification

Part of system: Login/Sign up

Pattern that governs part of system: Client – Server Pattern

Justification:

We would like to incorporate individual experiences for the user, and allow for the information/models that they are using to be saved in a database. The number of users, and saved models on the application can be scaled without performance issues on the system in regards to numerous AI generations.

Our entire team has worked on Client-Server products before, our familiarity with the system and what is needed for the DB will allow this section to be feasible within our project.

By having the models saved to a server we can reduce the number of times a user may leave the application, come back, and need to spend more time reuploading a image/GLB model or rewriting the AI prompt and having the pipeline / data-flow pattern we plan on incorporating for the upload component run more times than necessary. Reducing overall resources from being used and the cost of AI per prompt.

This pattern could still come across bottleneck issues with too many requests to the servers by users, we would also have to be aware of the client size local storage as we will be handling large amounts of data with our GLB files. We can handle this by having a set limit that gets returned to the user at a time.

One Pattern Rejected and Why: Layered / Pattern was rejected for this part of the system. We need a system where there is a server to store the user information to allow for multiple user applications at one time and an overall individual user experience.

Part of system: Main Screen (Switcher)

Pattern that governs part of system: Client – Server Pattern

Justification: The 3d models will need to be held on the server to ensure that they are saved somewhere. Without saving them to the server we would be reliant on users uploading their models each time they access the site which would create a large bottle neck on the system.

This solution will have a large storage overhead, but it saves the site from the intense bottleneck of having the user upload 3d models.

One Pattern Rejected and Why: a client storage model might be possible, but it would require a large amount of file uploads to the site especially when users are trying to

Part of system: Viewer

Pattern that governs part of system: Layered pattern, running on client. Presentation layer is canvas and the on-screen controls. Logic layer handles camera control and layer visibility. Data layer is a loaded model held in browser memory.

Justification: The viewer does not store or process data for the other components, and there is only one model taken from the switcher that turns the user's input into an updated 3D view. A clear difference between UI, viewing logic, and the loaded model data that matches the shape, and the calls can go down through the layers. The team has experience developing front end web apps and a little bit of WebGL knowledge. Rendering

occurs on the user's device by using the browser's 3D graphics (WebGL). Due to this, there isn't a need for a network round trip when the user interacts with the viewer.

As for the scalability of the component, each user's device does their own rendering. So, adding users does not render loads to our server. A potential risk could be the actual size of the model, so this could be fixed by setting a limit to the file size. The project being in a web browser means it could run on a laptop or phone, and since a GLB file can load in one request, it also keeps the models within the memory limits of the devices.

One Pattern Rejected and Why: The team rejected the client-server pattern, because the server would render frames (of the models) and project them to the browser which could mean lagging could be visible. This would also mean that we would need GPU servers that would be an issue due to budget constraints, and the server load would also grow for each user.

Part of system: Image Upload

Pattern that governs part of system: Pipeline / Data-Flow Pattern

Justification: This pipeline pattern for the image upload part of the system does match how the data and control must move as it will be uploaded from the user and to the backend and then to the screen in that flow. The team can realistically build and maintain this structure. With performance, the pattern will fit real time constraints and throughput. It is scalable as it will support the expected users on the hardware that our platform is intended for, which is web-based.

One Pattern Rejected and Why: The pattern that was rejected was the microservices pattern since this part is not that large, necessary for independent scaling.

Decision Log

Decision Log 1: Decided the necessity of a sign in/ log in component of the project, for security reasons and individual user experiences.

Decision Log 2: Decided to separate the 3D model viewer page and the main page, to allow for more user experience. This allows the user to save uploaded models and pick what model they would like to see. We chose this path over a plan WebGL where the user uploads their model views it, then loses their data when reopening the application.

Decision Log 3: A decision was made to have the switcher also handle the layers of the 3d models because it creates one cohesive component that is nice for the user to interact with.

Decision Log 4: Decided to render the 3D models on the client (user's browser) instead of the server because it would allow user's to interact with the model without network delays or GPU servers.